INEM · IA pour les systèmes embarqués · 2026–2027

Construire, entraîner et optimiser un petit réseau de prédiction sur système embarqué

Un réseau à mélange temporel et de variables, inspiré de TSMixer

Travaux dirigés et initiation à la recherche expérimentale · Roland Cédric TAYO

Résumé

Ce projet consiste à programmer un petit réseau de prédiction sans bibliothèque d'apprentissage. À partir de trois signaux simulés, le réseau utilise les seize derniers instants pour prédire une valeur future. Les étudiants écrivent les calculs, les gradients et l'apprentissage, puis déploient leur propre inférence sur Raspberry Pi. Ils étudient ensuite une optimisation d'une projection dense, vérifient la correction et mesurent son intérêt matériel. Le travail produit des courbes, des traces reproductibles et un rapport court au format scientifique. La progression est commune : calcul sur papier, fonction programmée, test, réseau complet, puis expérience sur Pi. Les ateliers des sections 8 et 9 guident les fonctions sans fournir leur code.

Mots-clés : prédiction ; réseau résiduel ; calcul embarqué ; mémoire ; optimisation ; reproductibilité.

1. Problème et objectifs

Un appareil doit prédire localement une mesure à partir de son historique, sans dépendre d'Internet. Sa mémoire et son temps de calcul sont limités. L'objectif est de comprendre comment un modèle apprend, puis comment son implémentation peut être adaptée à une petite cible. Les signaux du TD sont synthétiques : ils servent à vérifier une méthode, sans démontrer une application industrielle.



Figure 1. Le même modèle est appris sur PC, puis exécuté et mesuré sur la cible.

Question de recherche. À calcul et poids fixés, quelle organisation d'une projection dense réduit le temps de calcul ou la mémoire temporaire sur la cible, et dans quelles conditions cet avantage disparaît-il ?

Acquis attendus

- Expliquer poids, biais, activation, perte, gradient, apprentissage et inférence.
- Programmer et vérifier un réseau, séparer les données et interpréter ses erreurs.
- Déployer une inférence autonome ; distinguer poids, buffers, stockage et RAM du processus.
- Formuler une hypothèse, mesurer un compromis et rendre l'expérience reproductible.

Cadre. Quatre journées : 14 h de cours et 14 h de TD, puis approfondissement jusqu'en décembre selon le calendrier annoncé. Travail en équipes de 4 à 5 ; maîtrise individuelle évaluée. Prérequis : fonctions, boucles, tableaux, compilation, terminal, Git, produits de matrices et dérivées simples. Le storyboard de 3 h est une introduction ; les 14 h de cours et le travail personnel accompagnent les calculs ci-dessous.

Rappels A. Lire les données et les équations

Une équation est une recette. On remplace chaque symbole par sa valeur, puis on calcule. Ici, les signaux n'ont pas d'unité physique imposée. Un instant est un pas d'échantillonnage, pas nécessairement une seconde.

Symbole	Sens dans ce TD
s ; t	s est l'identifiant de séquence, de 0 à 23. t est l'indice de temps : 0 à 191 lors de la génération. t−1 signifie « l'instant précédent ».
x_1, x_2, x_3	Les trois valeurs observées à chaque instant. $x_c(t)$ est la valeur de la variable numéro c à l'instant t ; c vaut 1, 2 ou 3.
P ; φ ; π	P est la période entière tirée pour une séquence, entre 24 et 64. φ (phi) est un décalage de phase, en radians. π (pi) ≈ 3,14159. La période n'est pas un nombre de paramètres du réseau.
$\varepsilon_c(t)$	ε (epsilon) désigne un petit bruit aléatoire pour la variable c à l'instant t. Un nouveau tirage uniforme dans [−0,02 ; 0,02] est effectué pour chaque variable et instant.
sin ; cos ; max	Sinus et cosinus prennent ici un angle en radians. max(a,b) conserve la plus grande des deux valeurs. Ainsi max(0,−2)=0 et max(0,3)=3.
μ_c ; σ_c	μ (mu) est la moyenne de la variable c sur les observations d'apprentissage. σ (sigma) est son écart-type : il décrit l'amplitude de ses variations autour de cette moyenne.
N_{obs} ; r	N_{obs}=16×160=2 560 observations d'apprentissage par variable, avant fenêtrage. r parcourt ces observations, sans répéter les points présents dans plusieurs fenêtres.

$$\mu_c = (1/N_{obs}) \Sigma_{r=1}^{Nobs} x_{c,r},$$

$$\sigma_c = \sqrt{[(1/N_{obs}) \Sigma_{r=1}^{Nobs} (x_{c,r}-\mu_c)^2]},$$

$$\bar{\sigma}_c = \max(\sigma_c, 10^{-6}) ; \quad \tilde{x}_c = (x_c-\mu_c)/\bar{\sigma}_c.$$

Σ (sigma majuscule) signifie « additionner tous les termes ». √ est la racine carrée. Le carré ² multiplie une quantité par elle-même. La barre sur σ indique l'écart-type protégé contre une division par zéro ; le tilde sur x indique une valeur normalisée. Cette recette utilise le diviseur N_{obs}, et non N_{obs}−1.

La cible y est le prochain x_1. On entraîne avec $\tilde{y}=(y-\mu_1)/\bar{\sigma}_1$. La sortie normalisée $\tilde{\hat{y}}$ redevient une prédiction lisible par $\hat{y}=\bar{\sigma}_1\tilde{\hat{y}}+\mu_1$. Un chapeau signifie toujours « prédit » ; les métriques finales comparent ŷ et y sur l'échelle d'origine.

Erreurs de prédiction. MSE est l'erreur quadratique moyenne ; MAE, l'erreur absolue moyenne. Dans les formules de la section 2, N est le nombre de prédictions évaluées, i va de 1 à N, y_i est la cible et $\hat{y}_i$ sa prédiction. |a| donne la valeur absolue de a. On moyenne des erreurs, pas des pourcentages de bonnes classes.

2. Données : générer, séparer, prédire

Écrire un générateur de **24 séquences indépendantes**. Pour chaque séquence, tirer une période entière P entre 24 et 64 et une phase φ dans $[0, 2\pi[$. Initialiser les états précédents à zéro. Pour $t = 0,\ldots,191$, calculer :

$$x_1(t) = 0{,}60\, x_1(t-1) + 0{,}25\, x_2(t-1) + 0{,}15 \max(0, x_3(t-1)) + \varepsilon_1(t),$$

$$x_2(t) = \sin(2\pi t / P + \varphi) + \varepsilon_2(t),$$

$$x_3(t) = 0{,}70\, x_3(t-1) + 0{,}30 \cos(2\pi t / (P+7) + \varphi) + \varepsilon_3(t).$$

Les bruits ε sont indépendants et uniformes entre $-0{,}02$ et $+0{,}02$. Utiliser la graine **17 + identifiant de séquence**, de 0 à 23 ; nommer le générateur pseudo-aléatoire utilisé. Écarter les 32 premiers instants de chaque séquence et conserver les 160 suivants. Exporter un CSV par séquence : `t,x1,x2,x3`. Des langages différents peuvent produire des tirages différents : toute comparaison utilise exactement les mêmes CSV, dont les empreintes sont conservées.

2.1. Partition avant fenêtrage

Ensemble	Séquences	Usage
Apprentissage	0 à 15	Poids et statistiques de normalisation
Validation	16 à 19	Réglages et sélection du checkpoint
Test réservé	20 à 23	Évaluation finale après gel des choix

Créer ensuite les fenêtres à l'intérieur de chaque séquence. Une entrée contient 16 instants × 3 variables ; sa cible est x_1 à l'instant suivant. Il y a **144 exemples par séquence**, soit 2 304 exemples train, 576 validation et 576 test. Aucune fenêtre ne traverse deux séquences ou deux ensembles.

2.2. Normalisation et références

Calculer moyenne μ_c et écart-type σ_c sur les observations train de chaque variable ; remplacer σ par $\max(\sigma, 10^{-6})$. Normaliser les entrées avec ces seules statistiques. Normaliser la cible avec μ_1, σ_1 ; dénormaliser les prédictions pour les figures et les métriques finales.

Comparer obligatoirement le réseau à deux prédicteurs : **persistance**, soit la dernière valeur observée de x_1, et **régression linéaire** sur les 48 valeurs d'entrée. La seconde utilise la même perte et une descente de gradient, sans solveur externe. Ajouter un témoin analytique : appliquer l'équation de x_1 sans le bruit ε, aux dernières valeurs observées. Il utilise la connaissance du simulateur, pas un apprentissage ; discuter ce qu'apporte réellement le réseau.

$$\mathrm{MSE} = (1/N)\, \Sigma_i (\hat{y}_i - y_i)^2 \; ; \quad \mathrm{MAE} = (1/N)\, \Sigma_i |\hat{y}_i - y_i|.$$

À montrer. Tracer les trois signaux d'une séquence, puis valeurs réelles et prédites. Rapporter MSE et MAE sur l'échelle d'origine : ce problème de régression n'a pas un « taux de bonnes classes ». Une référence simple peut être meilleure que le réseau. Les équations du simulateur rendent aussi possible un prédicteur analytique ; discuter cette limite plutôt que revendiquer un besoin industriel d'IA.

3. Réseau à programmer

Le réseau conserve l'idée de mélange temporel et de variables de TSMixer [1,2]. Cette **adaptation pédagogique** comporte un seul bloc, sans normalisation interne, dropout ni attention. Elle ne reproduit pas l'architecture complète ou les performances de l'article.

3.1. Propagation avant

Noter $X \in \mathbb{R}^{16\times 3}$ la fenêtre normalisée et ReLU(a) = max(0,a), élément par élément. Pour chaque variable *c*, utiliser les **mêmes** matrices temporelles A et B :

$$h_c = \mathrm{ReLU}(A\,X_{:,c} + a)\ ; \quad U_{:,c} = X_{:,c} + B\,h_c + b.$$

Pour chaque instant *t*, utiliser les **mêmes** matrices C et D :

$$g_t = \mathrm{ReLU}(C\,U_{t,:}^{\top} + q)\ ; \quad Z_{t,:}^{\top} = U_{t,:}^{\top} + D\,g_t + d,$$

$$\hat{y} = v^{\top}\,\mathrm{vec}(Z) + \beta.$$

`vec` parcourt les instants, puis les variables. Les additions avec X et U sont les connexions résiduelles : elles transmettent l'entrée et une correction apprise.

Paramètres	Dimensions	Nombre
A, a ; B, b	8×16, 8 ; 16×8, 16	280
C, q ; D, d	8×3, 8 ; 3×8, 3	59
v, β	48, 1	49
Total	**388 paramètres entraînables**	**1 552 octets en float32**

3.2. Gradients et apprentissage

Écrire la propagation arrière par la règle de la chaîne. Pour une couche y = Wx+b et un gradient reçu δ : $\partial L/\partial W = \delta x^{\top}$, $\partial L/\partial b = \delta$ et $\partial L/\partial x = W^{\top}\delta$. ReLU transmet le gradient si son entrée est positive ; prendre une dérivée nulle en zéro. Une addition transmet le gradient aux deux branches. **Les paramètres partagés cumulent les contributions de tous leurs usages.**

Initialiser les poids uniformément entre −0,1 et +0,1 avec `random.Random(42)`, les biais à zéro. Parcours initial fixe des fenêtres, mini-lots de 16, perte quadratique moyenne, $\eta = 10^{-3}$, 20 époques au maximum. Remettre les accumulateurs de gradients à zéro à chaque lot. Le gradient de sortie vaut $2(\hat{y}-y)/m$ pour un lot de m exemples. Appliquer $\theta \leftarrow \theta - \eta\nabla L$ après le lot. Garder le meilleur checkpoint de validation. Après tests, un seul second essai à $\eta=10^{-4}$ est permis ; choisir sur validation, jamais sur test.

Vérifications. Tester une projection à la main ; contrôler les dimensions ; comparer des gradients analytiques et des différences finies centrées [3], en double précision sur quelques entrées, hors changement de branche de ReLU. Utiliser $g_{num} = [L(\theta+h)-L(\theta-h)]/(2h)$, avec $h = 10^{-5}$. Viser $|g-g_{num}| \le 10^{-6} + 10^{-3}\max(|g|,|g_{num}|)$. Contrôler deux coefficients par matrice et vecteur de biais, ainsi que β, sur deux petites entrées. Sur les 16 premières fenêtres train, vérifier une perte finie et plus faible après 20 époques qu'au départ.

Chronométrer deux époques ; garder durée, perte train et MSE validation. Si une valeur devient non finie, arrêter et vérifier normalisation, gradients et moyenne du lot. Si la perte ne baisse pas, refaire l'exercice (§9) et le contrôle numérique. Ne pas remplacer le modèle pour masquer un échec.

Rappels B. Lire le réseau et l'apprentissage

Symbole	Signification et dimensions
X ; U ; Z	Trois tableaux de 16 lignes (instants) et 3 colonnes (variables). X est l'entrée ; U est le résultat du mélange temporel ; Z est le résultat du mélange entre variables.
$\mathbb{R}$; ∈ ; ×	$\mathbb{R}$ désigne les nombres réels ; ∈ signifie « appartient à ». $X \in \mathbb{R}^{16\times 3}$ signifie « tableau réel de 16 lignes et 3 colonnes ». × sépare ici les dimensions.
A, a ; B, b	Poids et biais du mélange temporel : A de taille 8×16, a de taille 8 ; B de taille 16×8, b de taille 16. Les minuscules sont des vecteurs de biais.
C, q ; D, d	Poids et biais du mélange des variables : C de taille 8×3, q de taille 8 ; D de taille 3×8, d de taille 3. Le biais q est distinct de l'indice c, qui est un numéro de variable.
h_c ; g_t	Vecteurs cachés de 8 valeurs : h_c mélange les 16 instants de la variable c ; g_t mélange les 3 variables à l'instant t.
: ; ⊤ ; vec	« : » sélectionne toutes les lignes ou colonnes. ⊤ transpose un tableau (échange lignes et colonnes). vec(Z) range les 48 valeurs : $Z_{1,1}$, $Z_{1,2}$, $Z_{1,3}$, $Z_{2,1}$, etc.
v ; β ; $\tilde{y}$	v contient les 48 poids de sortie ; β (bêta) est le biais scalaire. $\tilde{y}$ est la valeur prédite normalisée. Dans les équations du réseau ci-contre, ŷ désigne cette sortie normalisée pour alléger l'écriture.
ReLU	Activation max(0,a), appliquée à chaque valeur. La lettre a dans cette définition représente une valeur quelconque, pas seulement le biais temporel.
θ ; L ; ∇ ; ∂	θ (thêta) regroupe les 388 paramètres. L est la perte. ∇L (nabla L) regroupe les dérivées de cette perte ; ∂ désigne une dérivée par rapport à une variable. Une dérivée indique comment une petite variation change la perte.
η ; δ ; W, x, y, b	η (êta) est le pas d'apprentissage. δ (delta) est le gradient reçu par une couche. Dans y=Wx+b, W, x, y, b sont les poids, l'entrée, la sortie et les biais d'une couche quelconque ; y n'est alors pas la cible du jeu de données.

Convention de lot. m=16 est le nombre d'exemples du mini-lot (éventuellement moins pour le dernier). Pour $L=(1/m)\Sigma_{i=1}^{m}(\tilde{y}_i-\tilde{y}_i)^2$, le gradient de sortie est $2(\tilde{y}_i-\tilde{y}_i)/m$. Additionner ensuite toutes les contributions aux paramètres partagés. **Ne pas diviser une seconde fois par m.** $\theta \leftarrow \theta-\eta\nabla L$ signifie « remplacer les poids par leurs valeurs corrigées ». Une époque est un passage sur tous les exemples d'apprentissage.

Contrôle numérique. $h=10^{-5}$ est une petite perturbation d'un seul coefficient de θ ; ce h n'est pas le vecteur h_c. g est sa dérivée analytique ; g_{num} est son approximation numérique. |a| est la valeur absolue de a : |−3|=3. Le signe ≤ signifie « inférieur ou égal » ; 10^{-6}=0,000001.

Temps et accélération. $T_{réf}$ et T_{var} sont les coûts de référence et de variante, strictement positifs et mesurés dans la même unité (secondes, millisecondes ou nanosecondes). Une accélération de 2 signifie deux fois moins de temps ; un gain de 50 % signifie une moitié du temps économisée. Une durée nulle à cause d'un timer trop grossier ne permet aucun ratio.

4. Réalisation et déploiement

Langages du TD : Python 3 sur PC, C sur Raspberry Pi. Bibliothèques standard autorisées ; Matplotlib pour tracer. Écrire les calculs avec des boucles, sans NumPy, BLAS, framework ni autodifférentiation dans le modèle. Seule l'inférence est portée en C, pas l'apprentissage. Pour chaque fonction : calculer un petit cas sur papier, coder, tester, puis expliquer son rôle. Les sections 8 et 9 donnent ces étapes.

4.1. Quatre TD de 3 h 30

Séance	Travail et temps indicatifs	Preuve à conserver
TD1 Fondations	25 min : besoin et dépôt ; 40 : signaux et partition ; 70 : calculs sur papier puis fonctions (§8) ; 45 : tests ; 30 : bilan.	Fiche de calcul, tests des fonctions et courbe des trois signaux.
TD2 Apprendre	70 min : petit gradient puis réseau (§9) ; 60 : apprentissage ; 40 : références et erreurs ; 25 : export/relecture ; 15 : bilan.	Poids, courbes, MSE des références et du réseau.
TD3 Embarquer	40 min : carte et compilation ; 55 : inférence ; 35 : vérification PC/cible ; 50 : mesures de référence ; 30 : hypothèse.	Inférence correcte sur carte, ressources et protocole daté.
TD4 Enquêter	60 min : une variante ; 35 : correction ; 45 : mesures ; 30 : effet global ; 25 : reproduction croisée ; 15 : conclusion.	Comparaison pilote, figure et prochaine expérience.

Passages autonomes. Avant d'assembler : tests des fonctions réussis ; avant d'entraîner : gradients vérifiés ; avant de mesurer : sorties PC/Pi dans la tolérance. Un pair vérifie chaque passage dans Git. En cas d'échec, corriger le premier test défaillant. Les mesures et le rapport se poursuivent à domicile ; vérifier l'accès au Pi avant TD3.

4.2. Contrat matériel

Cible du TD : Raspberry Pi sous Linux. Pour environ 35 étudiants et 14 Pi, prévoir sept groupes de cinq, un Pi par groupe au minimum. Il faut une alimentation adaptée, une microSD avec Raspberry Pi OS et un accès réseau/SSH ou écran/clavier. Avant TD1, vérifier démarrage, transfert et exécution. Noter modèle, RAM, architecture 32/64 bits, OS, CPU, compilateur et options. Python prépare et entraîne ; C exécute les prédictions sur Pi.

Déployer. Pousser le tag Git ; sur Pi, cloner/récupérer cette version et transférer poids, statistiques et fenêtres. Créer les commandes `make test-c`, `make inference` et `make benchmark` ; documenter leur lancement dans le README. Compiler sur Pi avec GCC ; exécuter localement, puis récupérer les CSV. SSH pilote, le Pi calcule. Ni capteur ni cloud requis ; ce réseau ne sature pas sa RAM.

Correction. Comparer toutes les fenêtres de validation sur Python PC, C PC et C Pi avant d'annoncer une performance. Tolérance sur chaque sortie : $|\hat{y}_{\text{carte}} - \hat{y}_{\text{PC}}| \leq 10^{-5} + 10^{-4}|\hat{y}_{\text{PC}}|$ sur les sorties normalisées. Si elle est dépassée, vérifier ordre des poids, type, normalisation et aplatissement ; ne pas élargir la tolérance pour faire passer le test. Sauvegarder puis recharger ne doit pas changer les prédictions.

5. Petite recherche : améliorer une brique

Socle commun. Étudier la projection dense y = Wx+b, réellement appelée par le réseau. Partir d'une référence scalaire correcte, compilée avec les mêmes options que la variante. Écrire avant les essais : « Nous pensons que […] réduira […], car […]. Cette hypothèse sera mise en défaut si […]. » Une seule hypothèse ; une variante suffit. Deux autres essais sont possibles après validation.

Levier possible	Modification étudiée	Contrepartie à rechercher
Parcours	Changer l'ordre des boucles ou la disposition des poids.	Préparation et lectures supplémentaires.
Réemploi	Calculer plusieurs sorties ensemble ; essayer un petit bloc.	Registres, taille du code, cas trop petits.
Buffers	Éviter des copies ou réutiliser un tampon dont la durée de vie est terminée.	Écrasement de données encore nécessaires.

Garder les mêmes poids et le calcul flottant : aucun réentraînement ni extension n'est demandé. Une seule variante correcte suffit ; expliquer aussi les essais défavorables.

5.1. De la brique au réseau

Mesurer les formes du modèle, puis les projections isolées 16→8, 32→16 et 64→32 avec des valeurs déterministes sauvegardées. Les deux dernières ne sont pas de nouveaux réseaux entraînés. Chronométrer séparément une éventuelle réorganisation des poids et discuter son amortissement.

Vérifier la brique, puis le réseau entier avec la fonction modifiée. Identifier la version appelée. Chercher où la variante aide ou échoue, sans imposer l'existence d'un gain.

5.2. Protocole fixé : PC, puis Pi

1. **Référence.** Même C, données, poids et options `-O2` pour les deux versions. Noter compilateur, OS, carte, charge, température et fréquence. La variante doit respecter la tolérance du §4 ; aucun gain minimal n'est exigé.
2. **Calibrer.** Sous Linux, utiliser clock_gettime(CLOCK_MONOTONIC). Échauffer 100 appels. Doubler le nombre K d'appels depuis 1 jusqu'à ≥100 ms ; plafond $K=2^{24}$, signaler s'il ne suffit pas. Garder K pour les deux versions d'une forme et d'une campagne.
3. **Répéter.** Trois campagnes de 10 paires référence/variante par forme, en inversant l'ordre à chaque paire. Varier les entrées de façon déterministe, sans allocation ni fichier dans le temps mesuré. Utiliser un checksum après le lot. Garder chaque durée et K dans le CSV ; temps par appel = durée/K.
4. **Comparer.** Tracer médiane et étendue min–max des 10 lots, sur PC puis Pi. Dire si la variante gagne dans les trois campagnes ; signaler les dispersions qui se chevauchent. Ce constat ne garantit ni un gain universel ni un temps réel.
5. **Compter.** Poids, buffers, fichiers et pic RSS du processus sont distincts. RSS inclut le runtime ; noter outil et unité. Mesurer préparation séparément et réseau entier. Sans instrument, énergie = non mesurée.

$$\text{Accélération} = T_{réf}/T_{var} \text{ ; gain (\%)} = 100 \times (T_{réf} - T_{var})/T_{réf}.$$

Décision. Une variante plus lente ou dans la dispersion peut obtenir tous les points : conserver, expliquer et proposer la suite. Aucun seuil n'est à demander à l'enseignant. Le comparateur optimisé [4] reste facultatif.

6. Règles, rendus et évaluation

Autorisé	Non autorisé dans le socle
Bibliothèque standard : fichiers, mathématiques, allocation, temps, aléatoire ; outils système et de compilation du Pi.	Framework de réseau, différentiation automatique, BLAS ou fonction externe réalisant à votre place projection, gradients, apprentissage ou inférence.
Outils de figures et de mise en page ; NumPy limité à l'analyse et aux figures ; tests écrits par l'équipe.	Utiliser NumPy ou un équivalent pour les calculs du modèle évalué ; importer un modèle ou moteur existant à la place de son implémentation.
Documentation, articles, petits extraits génériques attribués ; bibliothèque optimisée dans une expérience de comparaison clairement séparée.	Reprendre un réseau complet ou un corrigé ; dissimuler du code repris, modifier les données de test ou fabriquer des mesures.

Assistants IA. Aide autorisée pour comprendre, documenter, déboguer et proposer du code ou des hypothèses. Déclarer les usages substantiels et les parties concernées. Tout code remis doit être compris, testé et défendable ; une sortie d'IA n'est ni une source scientifique ni une preuve. Aucun détecteur ne remplace la vérification individuelle.

6.1. Rendus

Un dépôt Git relançable contient sources, tests, configuration, graines, données/poids ou procédure exacte de génération, versions et mesures brutes CSV. Un manifeste relie chaque campagne au commit, au binaire et à la carte. Une autre équipe relance un cas. Une commande de tests PC et une CI GitHub Actions sont obligatoires ; elles ne certifient pas les performances de la carte.

Le **rapport contient au maximum 15 pages au total**, références et annexes comprises. Viser 6 à 10 pages utiles. Il présente : résumé, problème et sources, méthode, protocole, résultats, limites et suite. Inclure trois figures : apprentissage, réel/prédit et optimisation sur Pi. La courbe des signaux et la fiche de calcul restent dans le dépôt. Ne pas imposer une amélioration positive. Un bilan individuel décrit la fonction et l'expérience maîtrisées. La soutenance individuelle comprend une explication et un petit changement prédit puis vérifié ; elle est organisée hors des 28 h. Chacun prédit puis vérifie une condition de mesure nouvelle annoncée à l'oral.

6.2. Barème sur 20

Collectif, 12 points : calculs/apprentissage corrects (3), déploiement et mesures (3), hypothèse et analyse des limites (3), DevOps, reproductibilité et clarté du compte rendu (3). Pour chaque critère : absent ou invalide, 0 ; partiel, moitié ; complet et vérifiable, totalité. **Individuel, 8 points :** expliquer une fonction et un résultat (4), prédire puis vérifier une modification (4). Un résultat négatif rigoureux peut obtenir tous les points.

Bonus de présentation : jusqu'à +2 points

+0,5 pour un rapport rédigé dans un anglais compréhensible ; +0,5 pour une présentation d'article scientifique soignée ; +1 pour un projet Overleaf gratuit recompilable avec ses sources, bibliographie et figures. **Note finale = min(20, note de base + bonus).** Aucun abonnement n'est requis. Les 20 points de base restent accessibles sans ces options ; la qualité scientifique et la traçabilité sont évaluées dans le socle.

Lire le document séparé *Consignes du rapport scientifique* et l'exemple anglais. Les annexes techniques volumineuses vont dans le dépôt, sans contourner la limite de pages. La remise suit le calendrier de décembre du module. Les choix techniques, tolérances et expériences du socle sont fixés dans ce sujet.

7. GitHub : rendre le travail visible et relançable

Dès le début : créer un dépôt GitHub par groupe, par exemple `inem-embedded-forecasting-g01`. Un dépôt privé convient. Inviter tous les membres et **rolln7drktayau** (enseignant) dans *Settings* → *Collaborators* → *Add people*. Vérifier l'acceptation avant le premier contrôle ; l'invitation seule ne prouve pas que l'accès fonctionne.

Le README donne les noms et comptes des membres, le lien du dépôt, le besoin étudié, la carte, les versions d'outils et les commandes exactes pour générer, tester, entraîner, compiler et exécuter sur Pi. **Les identifiants, fonctions, commentaires et messages de commit sont de préférence en anglais.** Expliquer les choix et les unités ; un commentaire qui répète une boucle ne suffit pas.

7.1. Premier envoi

Installer Git et s'authentifier selon sa documentation. Créer sur GitHub un dépôt vide. Dans votre dossier local contenant le README, adapter YOUR_ACCOUNT et gXX :

```
git init -b main
git add README.md
git commit -m "docs(project): describe objective and team"
git remote add origin https://github.com/YOUR_ACCOUNT/inem-embedded-forecasting-gXX.git
git push -u origin main
```

7.2. Une opération = un changement nommé et vérifiable

Créer une branche pour une tâche. Ajouter des commits cohérents et les pousser pendant le travail. Exemple, lorsque les dossiers `src` et `tests` existent :

```
git switch -c feat/dense-forward
git add src tests
git commit -m "feat(dense): implement scalar forward pass"
git push -u origin feat/dense-forward
```

Ouvrir une *pull request* vers `main`. Un autre membre relit, vérifie les tests et commente ; fusionner après correction. La branche principale reste utilisable. Puis récupérer la fusion et nommer l'étape validée :

```
git switch main
git pull --ff-only
git tag -a v0.2-core -m "Validated dense and gradient tests"
git push origin v0.2-core
```

Un commit enregistre localement ; `git push` envoie sur GitHub. Un tag désigne une version précise. Un push ne déploie pas le modèle sur le Pi : récupérer les fichiers, compiler et tester constituent une étape distincte.

7.3. Jalons et preuves attendues

Version poussée	État à retrouver dans GitHub
`v0.1-data`	Générateur, partition, statistiques train, figure des signaux et empreintes des CSV.
`v0.2-core`	Projection, activation, gradients et tests numériques qui passent.
`v0.3-trained`	Réseau, références, courbes, poids sauvegardés, configuration et rechargement vérifié.
`v0.4-pi-baseline`	Compilation sur Pi, équivalence PC/Pi et mesures de référence.
`v0.5-experiment`	Hypothèse datée, variante correcte, mesures brutes et analyse.
`v1.0-submission`	Version finale relançable : rapport, sources et bilan individuel.

Ces jalons sont des points de contrôle, pas un quota de commits. Conserver les étapes intermédiaires et les essais négatifs utiles. Ne pas reconstruire artificiellement l'historique à la fin. Exemples de branches : `feat/data-generator`, `feat/manual-backprop`, `feat/training-loop`, `feat/pi-inference`, `exp/loop-order`, `docs/paper`.

Organisation minimale à concevoir

```
README.md                # setup, commands, board, team
src/  tests/  configs/   # Python model, tests, settings
deployment/  Makefile     # C inference and Pi build targets
data/                    # small CSVs or exact generation recipe
artifacts/               # weights, normalization, checksums
experiments/             # hypothesis, manifests, raw CSVs
figures/  paper/         # plotting sources, PDF, LaTeX if used
.github/workflows/       # student-written CI workflow
```

Automatisation obligatoire. Écrire une commande de tests PC, puis une CI GitHub Actions qui la lance à chaque push et pull request. Tester au moins projection, gradients, sauvegarde/rechargement et un cas d'inférence connu. La CI prépare et vérifie le logiciel ; les temps du runner ne représentent pas les performances du Raspberry Pi.

Une mesure doit retrouver son origine

Chaque campagne possède un identifiant, le commit exact, une arborescence sans changements non enregistrés, les empreintes SHA-256 des données, des poids et du binaire, la configuration, la carte, la fréquence, le compilateur, ses options, le nombre K d'appels et l'unité de mesure. Conserver un CSV brut, par exemple `campaign,variant,shape,repeat,elapsed_ns,calls,checksum`. Indiquer explicitement le périmètre chronométré.

Conserver les petites données nécessaires ou une recette exacte avec versions et empreintes vérifiées. Ne pas commiter mots de passe, jetons, environnements virtuels ou fichiers de construction volumineux. Si le binaire est déposé en pièce jointe de release, le relier au tag et donner son empreinte. Un résultat sans fichier brut ni configuration ne constitue pas une preuve reproductible.

8. Atelier guidé A : comprendre chaque fonction

Règle commune : pour chaque ligne du tableau, écrire l'entrée, les étapes et le résultat sur papier ; programmer la fonction ; comparer ; conserver le test. Ne pas copier les 388 paramètres à la main. On travaille d'abord sur de petits nombres, puis on change les dimensions.

8.1. Préparer une entrée

Fonction à écrire	Petit exercice et preuve attendue
`generate_sequence`	Sans bruit, avec P=32 et φ=0, calculer t=0 puis t=1. Garder les trois anciennes valeurs avant la mise à jour. Comparer au code, puis rétablir le bruit et tracer les signaux.

`fit_scaler`
`normalize`
`denormalize`
	Calculer moyenne et écart-type de [1, 3], normaliser puis revenir aux valeurs initiales. Tester aussi [2, 2]. Le cas constant doit rester fini grâce au plancher de σ.

`make_windows`	Pour la liste [10, 20, 30, 40, 50] et une fenêtre de 2, écrire les entrées et cibles. Vérifier que la cible est le point suivant ; refaire avec les dimensions du TD.

8.2. Suivre une prédiction sur papier

Exemple réduit, distinct du réseau final : x=[1, −2], W=[[2, −1], [−1, 1]], b=[−1, 2], v=[½, −1], β=0, cible y=3. Chaque ligne de W est une sortie ; ½ vaut 0,5.

$$a=Wx+b \rightarrow r=\mathrm{ReLU}(a) \rightarrow z=x+r \rightarrow \hat{y}=v^{\top}z+\beta \rightarrow L=(\hat{y}-y)^2.$$

Fonction	À faire, puis à vérifier avec le programme
`dense_forward`	Développer les deux sommes de produits donnant a. Expliquer le rôle du biais ; changer un poids et prévoir quelle sortie change.
`relu_forward` `add_residual`	Calculer r puis z. Dire quelle valeur est annulée et ce que conserve l'addition. Tester séparément ReLU sur [−2, 0, 3].
`flatten` `predict_head`	Aplatir [[1, 2], [3, 4]] par lignes. Calculer ensuite la sortie ŷ de l'exemple réduit avec v, z et β.
`mse`, `mae`	Calculer les deux erreurs sur ce seul exemple, puis sur deux prédictions choisies. Vérifier une erreur nulle lorsque prédiction et cible sont égales.

Validation A. Fournir une fiche courte : fonction, entrées, résultat manuscrit, résultat du test, rôle dans le réseau. Chaque membre explique une fonction. Ensuite seulement, assembler les projections du Mixer avec les dimensions de la section 3.

9. Atelier guidé B : apprendre, vérifier, transférer

9.1. Comprendre une correction de poids

Reprendre l'exemple de la section 8, avec ses valeurs intermédiaires. Commencer par un seul exemple, donc m=1. Un gradient indique comment la perte change lorsqu'un paramètre varie légèrement.

1. **Sortie.** Calculer $\delta=2(\hat{y}-y)$, puis les gradients de v et β. La section 3 donne la dérivée de la projection : remplacer ses symboles par les nombres de votre fiche.
2. **Résidu et activation.** Revenir de z vers ses deux branches : le gradient est copié sur chacune. À travers ReLU, garder seulement la branche dont l'entrée a est positive.
3. **Projection.** Calculer les gradients de W et b, puis ceux de x. Pour x, additionner le chemin direct du résidu et celui qui traverse W.
4. **Mise à jour.** Écrire `sgd_step`. Avec $\eta=0{,}01$, modifier les paramètres une fois, recalculer la perte et constater son évolution. Les gradients sont calculés avant toute modification des poids.

Écrire `dense_backward`, `relu_backward` et le retour du résidu ; comparer au calcul papier. Contrôler ensuite les dérivées par différences finies (§3). Ne pas tester au point de cassure de ReLU.

9.2. Passer du petit exemple au réseau

Étape	Consigne et preuve avant de poursuivre
Poids partagés	Appeler deux fois la même projection sur deux entrées. Additionner ses contributions de gradient ; vérifier numériquement leur somme. Puis appliquer ce principe aux 3 variables et aux 16 instants.
Boucle d'apprentissage	Écrire `train_epoch` : gradients à zéro, avant, arrière, mise à jour après le lot. Vérifier la baisse sur les 16 premières fenêtres (§3) ; pour m=16, moyenner une seule fois.
Courbes et références	Enregistrer époque, perte train, MSE validation et durée. Tracer les pertes ; comparer la prédiction au dernier point observé et au modèle linéaire. Réserver le test jusqu'au gel des choix.
Exporter et relire	Écrire `save_model` et `load_model`. Fixer ordre des poids, dimensions, type numérique et statistiques. Vérifier la même sortie après relecture.
Python → C → Pi	Porter seulement la propagation avant. Tester d'abord le petit calcul en C, puis les fenêtres fixées avec les poids exportés. Comparer Python PC, C PC et C Pi avant les chronométrages.

9.3. Du test à l'expérience

Chaque fonction validée devient un test automatique. Ajouter cas nul, dimensions invalides et sorties impaires pour la variante. La CI exécute les tests PC ; elle ne mesure pas le Pi. Sur Pi, conserver une référence inchangée, modifier une brique, vérifier les sorties puis mesurer selon le §5.

Validation B. Montrer un calcul de gradient, la courbe d'apprentissage, l'accord PC/Pi et un CSV de mesure. Chaque étudiant prédit l'effet d'un petit changement avant de le tester. Un résultat nul ou défavorable est recevable ; les traces, la compréhension et l'analyse font la preuve.

10. Outils et sources : quoi télécharger ?

Il n'y a aucun modèle préentraîné ni jeu de données à télécharger. Vous générez les signaux de la section 2 et entraînez vos 388 paramètres. Aucun capteur, GPU, compte cloud payant ou environnement fourni n'est nécessaire. Le matériel minimal par groupe est un PC, un Raspberry Pi alimenté, une microSD avec OS et un accès réseau ou écran/clavier.

Outil ou ressource officielle	Usage et lien
Git et GitHub	git-scm.com/downloads ; installation et authentification ; inviter les collaborateurs.
Python 3 sur PC	python.org/downloads. random pour les tirages, csv pour les fichiers ; standard math pour sinus, cosinus et racine. Noter la version et le générateur.
C et compilation	GCC et GNU Make. Écrire les cibles du Makefile. Compiler référence et variante avec les mêmes options, par exemple `-O2` ; noter la version et les options exactes.
Outils de la carte	Raspberry Pi Imager et installation officielle ; SSH et transfert. Installer Git, GCC et Make sur Pi ; tester une exécution avant TD3.
Figures et CI	Matplotlib, autorisé pour les figures ; GitHub Actions, pour créer votre automatisation. Pas de NumPy dans le modèle évalué.
Article et LaTeX	Overleaf ; premier projet et import ZIP ; classe elsarticle. Le ZIP fourni contient un gabarit et un exemple, pas une solution du TD.

Lectures scientifiques : comprendre, pas recopier

[1] Chen et al., TSMixer (2023) et code des auteurs : architecture complète, distincte de notre petit réseau.
[2] Google Research : présentation illustrée de TSMixer.
[3] Stanford CS231n : contrôle des gradients.
[4] OpenBLAS : bibliothèque optimisée, uniquement comme comparateur séparé ; comparer un calcul et un nombre de threads équivalents.
[5] Jacob et al. (2018) : calcul entier et quantification, extension facultative.

Vocabulaire : glossaire Embedded France–Eclipse : données (§2), optimisation (§3), outils et matériel (§5–6), mesures (§9), versions et déploiement (§11). À consulter selon le besoin, pas à apprendre intégralement.

Première action concrète : vérifier le modèle de carte et les outils, créer le dépôt, inviter l'enseignant, écrire le générateur, afficher une séquence et expliquer la cible à prédire. Ne commencer l'optimisation qu'après les tests et la référence sur carte.

Petit lexique. CPU = processeur de calcul ; RAM = mémoire de travail ; microSD = stockage conservé hors alimentation. OS = système d'exploitation ; SSH = accès distant au terminal. RSS = mémoire physique occupée par un processus ; un buffer est un tableau temporaire. Le cache rapproche des données du CPU. CI = intégration continue ; CSV = fichier de valeurs séparées par des virgules. SHA-256 produit une empreinte qui permet de vérifier qu'un fichier n'a pas changé.